

Exploratory Data Analysis

Ask the Right Questions Before Building Any Model

■ EDA FOR DATA SCIENCE

■ LEARNING OUTCOME

By the end of this PDF you will conduct a rigorous EDA: understand data shape, detect patterns and anomalies, explore relationships between variables, and generate hypotheses that guide modeling decisions.

■ Skills You Will Gain

- Follow the 7-step EDA framework
- Profile a dataset with automated tools (pandas-profiling)
- Analyze univariate, bivariate, and multivariate distributions
- Detect and treat outliers with multiple methods
- Build a correlation heatmap and interpret it
- Write a structured EDA findings report

■ Great models start with great EDA. Skipping EDA is like performing surgery without reading the X-ray. Every pattern you miss in EDA becomes a bug in your model.

SECTION 1

The 7-Step EDA Framework

1	Shape & Types df.shape, df.info(), df.dtypes — know your dimensions and column types
2	Missing Values df.isnull().sum() → visualize with msno.matrix() → decide: drop or impute
3	Target Analysis Understand the variable you're predicting — distribution, class imbalance
4	Univariate Each feature individually: histogram (numeric), bar chart (categorical)
5	Bivariate Feature vs target: scatter, box plot, grouped bar, point-biserial correlation
6	Multivariate Correlation heatmap, pair plot, PCA scatter — find redundant features
7	Report Insights Document every finding: observation + chart + implication for modeling

SECTION 2

Automated Dataset Profiling

```
# pandas-profiling – generates full HTML report in 1 line pip install ydata-profiling from
ydata_profiling import ProfileReport profile = ProfileReport(df, title='Dataset EDA Report',
explorative=True) profile.to_file('eda_report.html') # open in browser # sweetviz – beautiful
side-by-side comparison pip install sweetviz import sweetviz as sv report = sv.analyze(df)
report.show_html('sweetviz_report.html')
```

SECTION 3

Univariate Analysis Code

```
import matplotlib.pyplot as plt import seaborn as sns sns.set_theme(style='whitegrid') # Numeric:
histogram + KDE overlay fig, axes = plt.subplots(2, 3, figsize=(15, 8)) numeric_cols =
df.select_dtypes(include='number').columns for i, col in enumerate(numeric_cols[:6]):
sns.histplot(df[col], kde=True, ax=axes[i//3][i%3], color='#7C3AED')
axes[i//3][i%3].set_title(f'{col} Distribution') plt.tight_layout() # Categorical: bar chart
categ_cols = df.select_dtypes(include='object').columns for col in categ_cols:
df[col].value_counts().plot(kind='bar', color='#06B6D4') plt.title(f'{col} Frequency')
plt.tight_layout() plt.show()
```

Bivariate Analysis

```
# Numeric vs Numeric – scatter + regression line sns.scatterplot(x='StudyHours', y='ExamScore',
data=df, hue='Gender') sns.regplot(x='Income', y='Spend', data=df) # with trend line # Numeric vs
Categorical – box plot / violin sns.boxplot(x='Department', y='Salary', data=df)
sns.violinplot(x='Grade', y='Score', data=df, palette='Set2') # Categorical vs Categorical –
crosstab heatmap ct = pd.crosstab(df['Region'], df['Segment']) sns.heatmap(ct, annot=True,
fmt='d', cmap='YlOrRd') # Target vs feature – compare distributions for col in numeric_cols:
sns.kdeplot(df[df['Churn']==0][col], label='No Churn') sns.kdeplot(df[df['Churn']==1][col],
label='Churned') plt.legend(); plt.title(col); plt.show()
```

SECTION 5

Correlation & Multivariate

```
# Correlation heatmap fig, ax = plt.subplots(figsize=(12,10)) corr =
df.select_dtypes(include='number').corr() mask = np.triu(np.ones_like(corr, dtype=bool)) # hide
upper triangle sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, mask=mask,
square=True, ax=ax) plt.title('Feature Correlation Matrix') # Pair plot (all numeric features vs
each other) sns.pairplot(df[numeric_cols[:5]+'target'], hue='target', diag_kind='kde') # Find
highly correlated pairs high_corr = [(c1,c2,corr.loc[c1,c2]) for c1 in corr.columns for c2 in
corr.columns if c1<0.85] print('Highly correlated pairs:', high_corr)
```

SECTION 6

Outlier Detection

```
# Box plots reveal outliers visually sns.boxplot(data=df[numeric_cols], orient='h',
palette='Set1') # IQR method Q1=df.quantile(0.25); Q3=df.quantile(0.75); IQR=Q3-Q1 outlier_mask =
((df>Q3+1.5*IQR).any(axis=1)) print(f'{outlier_mask.sum()} outlier rows detected') # Isolation
Forest (ML-based outlier detection) from sklearn.ensemble import IsolationForest iso =
IsolationForest(contamination=0.05, random_state=42) df['anomaly'] =
iso.fit_predict(df[numeric_cols]) # -1 = outlier
```

■ PRACTICE QUESTIONS

Q1. What is the purpose of EDA before building a model?

Answer: To understand data structure, detect quality issues, find patterns, and form hypotheses — so you build the right model with the right features.

Q2. What does a right-skewed distribution of income tell you for modeling?

Answer: The mean is pulled up by a few very rich people. Use median for central tendency. Consider log-transforming income before feeding it to a model.

Q3. You find correlation=0.91 between 'TotalRooms' and 'TotalBedrooms'. What should you do?

Answer: They're near-perfectly correlated (multicollinear). Drop one — keeping both adds noise and hurts model interpretability.

Q4. What is class imbalance and why does it matter for EDA?

Answer: When one target class vastly outnumbers others (e.g. 95% non-fraud, 5% fraud). A model predicting all 'non-fraud' gets 95% accuracy but is useless. Detect it in EDA to plan SMOTE/resampling.

Q5. What tool generates an automatic HTML EDA report in one line?

Answer: ydata-profiling (formerly pandas-profiling): ProfileReport(df).to_file('report.html')

■ EDA Cheat Sheet

<code>df.describe()</code>	Summary stats: mean, std, Q1, Q2, Q3
<code>df.isnull().sum()</code>	Missing count per column
<code>df['col'].value_counts()</code>	Category frequencies
<code>df.corr()</code>	Correlation matrix
<code>sns.histplot(df['col'], kde=True)</code>	Distribution histogram
<code>sns.boxplot(x, y, data=df)</code>	Box plot by category
<code>sns.heatmap(corr, annot=True)</code>	Correlation heatmap
<code>sns.pairplot(df, hue='target')</code>	All feature pair plots
<code>sns.scatterplot(x, y, hue)</code>	Colored scatter plot
<code>pd.crosstab(a, b)</code>	Category cross-tabulation
<code>IsolationForest</code>	ML-based outlier detection
<code>ProfileReport(df)</code>	Auto EDA HTML report